

System Design — Day 5

Topic: Full Design — URL Shortener (6-step framework)

1. The Framework

Requirements → Back-of-envelope math → API design → High-level architecture → Deep dive → Trade-offs.

2. Requirements (ask before designing)

Requirements means asking the interviewer questions *before* drawing anything. Examples:

- Can users choose custom short links?
- Do links expire? (Never assume yes. Plan for the worst case: links live forever.)
- How many URLs are created per day, and how many redirects?

Short and long URLs are one mapping stored together (short_code → long_url), not separate servers. App servers stay stateless (Day 1) and never talk to each other.

3. Read-Heavy Insight

A short link is created once but clicked many times. Standard assumption: about **100 reads for every 1 write**. A URL shortener is a **read-heavy** system, which means the design centers on **caching (Redis) and replicas**.

4. Back-of-Envelope Math

Assumptions: 100 million new URLs/month, 100:1 read:write, about 2.5 million seconds in a month, 500 bytes per record, kept 5 years.

Quantity	Calculation	Result
Writes per second	$100\text{M} \div 2.5\text{M}$	~40/s (trivial)
Reads per second	40×100	~4,000/s average, ~10,000/s at peak
URLs in 5 years	$100\text{M} \times 60$	6 billion
Storage	$6\text{B} \times 500 \text{ bytes}$	~3 TB (modest)

Conclusion: writes are trivial, storage is modest, and **reads are the real challenge**. That is why Redis and replicas exist. Tip: cancel zeros first to avoid magnitude errors.

5. Short Code Length

Each position has 62 possible characters (26 lowercase + 26 uppercase + 10 digits).

Length	Combinations
5 characters	~916 million
6 characters	~56 billion
7 characters	~3.5 trillion

6 characters covers the 6 billion needed (about 11% of the space used after 5 years). Many designs choose 7 for safety. Justify the choice with numbers, not with expiry assumptions.

6. Avoiding Duplicate Codes (Race Condition)

Two servers can generate the same random code at the same instant. Both check the database, both see "free," and both save. This is a **race condition**: the check and the save are two separate steps, and another server slips in between them.

Fix: put a **unique constraint / primary key** on **short_code** (not on the long URL). The database performs the check and save as one atomic step, so only one write can win. The loser catches the error, generates a new random code, and retries. The user never sees the collision.

7. API Design

- **POST /shorten** with {"long_url": ...} returns {"short_url": "short.ly/k9Xa2P"}
- **GET /k9Xa2P** returns an HTTP redirect to the long URL

301 = permanent; browsers cache it, which lowers load but blocks click counting. **302** = temporary; every click returns through your servers, so use it when analytics are needed.

8. Architecture

Redirect (read) path:

1. Browser sends GET to the load balancer.
2. Load balancer picks a stateless app server (it never talks to Redis or the database itself).
3. App server asks Redis for the short code.
4. Cache hit: return the long URL immediately.
5. Cache miss: read from a database replica, then save the result into Redis.
6. App server returns the 301/302 redirect; the browser goes to the long URL.

Create (write) path:

1. POST reaches the load balancer, which picks an app server.
2. App server generates a random 6-character code.
3. It writes to the **primary** database (never a replica).
4. If the unique constraint rejects the code, generate a new one and retry.

5. On success, return the short URL. The primary replicates to replicas automatically (brief replication lag is a mild issue here).

9. Trade-offs

- **Cache only hot links.** A small share of links gets most clicks. Redis memory is limited, so it evicts the least recently used (LRU) entries.
- **If Redis crashes,** all reads fall through to the database. Our ~4,000-10,000 reads/s exceeds what one database comfortably handles, so it could overload. (A single primary-key lookup is cheap; the danger is the volume.)
- **Protection:** run Redis replicas or a cluster, and warm the cache up gradually after a crash.

10. Key Takeaway

The math justifies every component: reads (not writes or storage) are the bottleneck, so caching and replicas are required. Correctness at write time comes from a database-level unique constraint, not from checking in application code.